Erkennung ungenutzter Bibliotheken in Deployments zur risikobasierten Priorisierung von Sicherheitsbefunden

Projektarbeit 1 (T4_1000)

Im Rahmen der Prüfung:

Bachelor of Science (B. Sc.)

des Studienganges Informatik

an der
Dualen Hochschule Baden-Württemberg Karlsruhe

von

Abgabedatum	05.01.2026
Bearbeitungszeitraum	20.10.2025 - 05.01.2026
Matrikelnummer, Kurs	, TIN
Ausbildungsfirma	SAP SE Dietmar-Hopp-Allee 16 69190 Walldorf, Deutschland
Betreuer der Ausbildungsfirma	
Gutachter der Dualen Hochschule	—

Eidesstattliche Erklärung

Ich versichere hiermit, dass ich meine Projektarbeit 1 (T4_1000) mit dem Thema:

> *Erkennung ungenutzter Bibliotheken in Deployments zur risikobasierten Priorisierung von Sicherheitsbefunden*

gemäß § 5 der „Studien- und Prüfungsordnung DHBW Technik“ vom 29. September 2017 selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

K, 02. März 2026

gez.: ,

Sperrvermerk

Die nachfolgende Arbeit enthält vertrauliche Daten der:

SAP SE
Dietmar-Hopp-Allee 16
69190 Walldorf
Deutschland

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anderslautende Genehmigung vom Dualen Partner vorliegt.

Erklärung zur Nutzung Künstlicher Intelligenz

Für die folgende Arbeit wurde die Unterstützung Künstlicher Intelligenz genutzt.
Gemini 2.5 Flash und Gemini 2.5 Pro wurden genutzt, um bei der Literatur Recherche und Fachterminologie Suche zu unterstützen und Rechtschreibung, Grammatik und Ausdruck zu verbessern.
Obwohl diese Hilfsmittel meine Fähigkeiten erweitert und zu meinen Erkenntnissen beigetragen haben, ist es wichtig darauf hinzuweisen das sie inhärente Einschränkungen aufweisen. Ich habe mich bemüht jegliche Unterstützung Künstlicher Intelligenz transparent und verantwortungsvoll zu nutzen.

Diese Erklärung wurde von mir übersetzt, jedoch automatisch von AI Usage Cards generiert. https://ai-cards.org

Abstract

- English -

Abstract1

Abstract

- Deutsch -

Abstract2

Inhaltsverzeichnis

Abbildungsverzeichnis ..... **IX**

Tabellenverzeichnis ..... **X**

1 Einleitung **1**
1.1 Motivation 1
1.2 Problemstellung und Forschungsfragen 2
1.3 Zielsetzung und Abgrenzung 2
1.4 Aufbau der Arbeit 2

2 Grundlagen **3**
2.1 Dependency Management 3
2.2 Software Bill of Materials (SBOM) 4
2.3 Bloated Dependencies 4
2.3.1 Definition und Abgrenzung 4
2.3.2 Kategorisierung 5
2.3.3 Entstehungsmechanismen 5
2.3.4 Empirische Evidenz 6
2.3.5 Konsequenzen 7
2.4 Vulnerability-Datenbanken und Dependency Scanner 7
2.4.1 CVE und die National Vulnerability Database 7
2.4.2 Von der Datenbank zum Scanner-Befund 8

3 Verwandte Arbeiten **10**
3.1 Dynamische Ansätze (Runtime-Tracing, Coverage) 10
3.2 Statische Ansätze (Call-Graph, Reachability) 10
3.3 Einordnung der Ansätze nach L0–L2 und Eignung für TypeScript/Node.js 10

4 Konzept und Designentscheidung **11**
4.1 Usage-Evidence-Schema 11
4.1.1 Terminologie 13
4.1.2 Formaler Rahmen und Geltungsbereich 14
4.2 Anforderungen an den PoC 19
4.3 Bewertung und Methodenauswahl 19
4.4 Architektur der Gesamtpipeline 19

5 Implementierung **20**
5.1 Tooling-Stack und Projektstruktur 20
5.2 L0: Präsenzermittlung via Dependency-Tree/SBOM 20
5.3 L1: Loaded-Heuristiken 20
5.4 L2: Reachability-Analyse (Call-Graph, Entry Points, Mapping) 20
5.5 Ergebnisformat und Risk-Scoring 20
5.6 Beispiel-Durchlauf 20

6 Evaluation **21**
6.1 Setup und Testkorpus 21
6.2 Metriken und Vorgehen 21

6.3 Ergebnisse und Vergleich mit npm audit 21
6.4 Limitierungen (JS-Dynamik, Reflection, Bundling) 21

7 Fazit und Ausblick 22
7.1 Fazit 22
7.2 Ausblick 22

Literaturverzeichnis ..... a

Abkürzungsverzeichnis

***CPE* – Common Platform Enumeration** 8

***CVE* – Common Vulnerabilities and Exposures** 7, 8, 9

***CVSS* – Common Vulnerability Scoring System** 8, 9

***CWE* – Common Weakness Enumeration** 8

***DI* – Dependency Injection** 14, 15, 16, 18

***FN* – False Negative** 14, 17, 18

***FP* – False Positive** 11, 14, 17, 18

***IoC* – Inversion of Control** 14

***NIST* – National Institute of Standards and Technology** 7

***NVD* – National Vulnerability Database** 7, 8

***OWASP* – Open Worldwide Application Security Project** 8, 11

***PoC* – Proof of Concept** 9, 11, 13, 17, 18

***ReDoS* – Regular Expression Denial of Service** 17

***SBOM* – Software Bill of Materials** 8, 17, 18

***SCA* – Software Composition Analysis** 8, 12

***TS* – TypeScript** 5

***npm* – Node Package Manager** 8, 15, 17

Abbildungsverzeichnis

Abbildung 1 Dreistufiger Prüfprozess eines metadatenbasierten Dependency-Scanners (Inventarisierung, Matching, Meldung). 9
Abbildung 2 Trichtermodell: Jede Evidenzstufe beschreibt eine echte Teilmenge der vorherigen. 14

Tabellenverzeichnis

Tabelle 1 Bloated Dependencies im Maven-Ökosystem [1]. 6
Tabelle 2 Bloated Dependencies im CommonJS/npm-Ökosystem [2]. 6
Tabelle 3 Evidenzstufen des Usage-Evidence-Schemas als Überblick. L1 wird in dieser Arbeit erstmals als eigenständige Sicherheitsstufe formalisiert. Die zugehörige Messung erfolgt in Kapitel 6. L2b erfordert eine kontrollierte Laufzeitumgebung; seine Umsetzung wird in Kapitel 7 als Gegenstand künftiger Forschung diskutiert. 18

1 Einleitung

1.1 Motivation

Dependency Scanner wie npm audit und OWASP Dependency-Check sind aus der modernen Softwareentwicklung nicht mehr wegzudenken [3]. Diese Tools basieren primär auf Software Bill of Materials (SBOM) und operieren ausschließlich auf Package-Ebene [4]. Aktuelle Forschung zeigt jedoch fundamentale Limitierungen dieser Ansätze, die zu erheblichen praktischen Problemen führen.

Die Unfähigkeit zu erkennen, ob Code tatsächlich zur Laufzeit geladen wird, führt zu zwei kritischen Problemen. Erstens akkumulieren Projekte obsolete Dependencies, die erhebliche Wartungskosten verursachen [5]. Empirische Analysen zeigen, dass 75,1% aller Maven-Dependencies als „bloated“ klassifiziert werden können, wobei 57% der transitiven Dependencies vollständig ungenutzt sind [6]. Im JavaScript-Ökosystem ist die Situation vergleichbar: 50,6% der Dependencies in CommonJS-Packages sind bloated [2].

Diese obsoleten Dependencies entstehen durch verschiedene Mechanismen. Transitive Abhängigkeiten werden automatisch in die Dependency-Hierarchie eingefügt, ohne dass Entwickler sich deren Präsenz bewusst sind [5]. Darüber hinaus werden Dependencies während der Entwicklung hinzugefügt und bei Code-Refactorings nicht entfernt [2]. Das Entfernen ist jedoch oft schwierig, weil Entwickler nicht mit ausreichender Sicherheit beurteilen können, ob eine Dependency wirklich entbehrlich ist [5]. Die resultierenden Kosten manifestieren sich in erhöhten Binary-Größen, verlängerten Build-Zeiten, erhöhtem Speicherverbrauch und gesteigertem Sicherheitsrisiko durch eine vergrößerte Angriffsfläche [6], [7]. In containerisierten Deployments wird die Problematik besonders sichtbar, da Images teils eine große Menge veralteter oder verwundbarer JavaScript-Pakete enthalten [8].

Zweitens führt die Fokussierung auf Package-Level-Analysen zu einer hohen Rate an False-Positive-Warnungen. Die zentrale Herausforderung liegt in der fehlenden Granularität: SBOM-basierte Tools können nicht zwischen deployed und nicht-deployed Code differenzieren [9], noch können sie feststellen, ob vulnerable Funktionen tatsächlich vom Anwendungscode aufgerufen werden.

1.2 Problemstellung und Forschungsfragen

1.3 Zielsetzung und Abgrenzung

1.4 Aufbau der Arbeit

2 Grundlagen

2.1 Dependency Management

Modern entwickelte JavaScript- und TypeScript-Anwendungen stützen sich in hohem Maße auf externe Bibliotheken. Um diese zu verwalten, verwenden Entwickler den Package Manager npm (Node Package Manager). Ein Package Manager übernimmt drei zentrale Aufgaben: Er liest Manifest-Dateien wie `package.json`, lädt die deklarierten Bibliotheken aus dem zentralen npm-Registry herunter und installiert sie lokal im Projekt [10]. Dabei löst er nicht nur die explizit angeforderten Dependencies auf, sondern rekursiv auch alle deren Abhängigkeiten.

Dependencies lassen sich in zwei Kategorien einteilen. Direkte Dependencies werden explizit im Projekt-Manifest deklariert, beispielsweise `express` in der `package.json`. Der Anwendungscode importiert und nutzt diese Bibliotheken direkt. Transitive Dependencies hingegen sind Abhängigkeiten dieser direkten Dependencies [1]. Wenn ein Node.js-Projekt `express@4.18.2` deklariert, installiert npm automatisch auch die 31 Dependencies von express, darunter `body-parser`, `cookie` und `debug`. Alle installierten Packages werden im `node_modules`-Verzeichnis abgelegt.

Dieser Mechanismus führt zum Phänomen der Dependency Amplification: Eine einzelne direkte Dependency zieht eine Kaskade weiterer Dependencies nach sich [11]. Ein Projekt mit 50 direkten Dependencies kann so auf mehrere hundert installierte Packages anwachsen [11].

Bei TypeScript-Projekten wird diese Amplification zusätzlich verstärkt: Für jede JavaScript-Bibliothek ohne eingebaute Typdefinitionen muss ein separates Type-Definition-Package aus dem `@types/*`-Namespace installiert werden. Nutzt ein Projekt beispielsweise `express`, muss zusätzlich `@types/express` installiert werden, um TypeScript-Typen zu erhalten. npm unterscheidet zwischen `dependencies` (zur Laufzeit benötigt) und `devDependencies` (nur für Build/Test). Type-Definition-Packages werden als `devDependencies` deklariert, da sie ausschließlich zur Compile-Zeit erforderlich sind.

Diese Dependency Amplification führt zu einem fundamentalen Problem: Bei hunderten automatisch installierten Packages verlieren Entwickler den Überblick, welche Dependencies tatsächlich vom Anwendungscode genutzt werden. Die in Abschnitt 1.1 beschriebenen Konsequenzen (akkumulierte bloated dependencies, erhöhte Sicherheits-

risiken und False-Positive-Warnungen in Vulnerability Scannern) sind direkte Folgen dieses Mechanismus.

2.2 Software Bill of Materials (SBOM)

Wie in der Motivation beschrieben, basieren Scanner wie npm audit oder OWASP Dependency-Check auf Software Bills of Materials (SBOMs). Ein SBOM ist eine maschinenlesbare Liste aller Softwarekomponenten eines Produkts [12]. Im Gegensatz zu Manifest-Dateien wie `package.json`, die nur direkte Dependencies auflisten, erfasst ein SBOM auch alle transitiven Dependencies mit exakten Versionen. Die NTIA spezifiziert Mindestinhalte [13], das verbreitetste Format ist SPDX (ISO/IEC 5962:2021) [14].

SBOMs werden typischerweise durch spezialisierte Tools wie CycloneDX, Syft oder das Microsoft SBOM Tool aus den Dependency-Informationen eines Projekts generiert. Jeder SBOM-Eintrag dokumentiert eine Softwarekomponente mit folgenden Informationen: Paketname (z.B. `express`), Version (`4.18.2`), eindeutiger Identifikator als Package-URL (`pkg:npm/express@4.18.2`), Lizenzinformation (`MIT`), Lieferant sowie die Liste der direkten Dependencies dieser Komponente. Für ein typisches Node.js-Projekt mit 50 direkten Dependencies kann ein SBOM mehrere hundert Einträge umfassen, da auch alle transitiven Dependencies erfasst werden müssen.

Die fundamentale Einschränkung von SBOMs liegt im Detailgrad der Dokumentation: Dependencies werden auf Package-Ebene dokumentiert. Eine Bibliothek wird als Ganzes erfasst, unabhängig davon, welche ihrer Funktionen tatsächlich genutzt werden. Bindet ein Projekt die Bibliothek `lodash` ein und nutzt nur die Funktion `_.debounce()`, erfasst das SBOM lediglich `lodash@4.17.21` als Dependency, ohne zu dokumentieren, welche der über 300 verfügbaren Funktionen von lodash im Code aufgerufen werden.

2.3 Bloated Dependencies

2.3.1 Definition und Abgrenzung

Als **bloated** werden Dependencies bezeichnet, die im Deployment einer Anwendung installiert sind, aber zur Laufzeit nie verwendet werden [1]. Bestehende Analyse-Tools fokussieren primär auf veraltete (*outdated*) und verwundbare (*vulnerable*) Dependencies. Eine Dependency kann jedoch veraltet sein, ohne ungenutzt zu sein, und eine

verwundbare Dependency muss nicht zwingend bloated sein. Diese Arbeit ergänzt die bestehenden Dimensionen um den Aspekt der genutzten Codebasis.

2.3.2 Kategorisierung

Bloated Dependencies lassen sich nach ihrer Herkunft in zwei für npm relevante Typen einteilen. **Direkte** bloated Dependencies sind explizit im Manifest deklariert, werden jedoch nie vom Anwendungscode aufgerufen. **Transitive** bloated Dependencies wurden automatisch als Abhängigkeiten anderer Packages installiert, ohne dass die Anwendung sie je verwendet [1]. Transitive bloated Dependencies sind in der Praxis dominierend, da der Dependency-Auflösungsmechanismus sie automatisch und ohne direktes Zutun der Entwickler in den Abhängigkeitsbaum einfügt (vgl. Abschnitt 2.1). Im Maven-Ökosystem existiert zusätzlich die Kategorie **geerbter** Dependencies aus Parent-POMs, die in *TypeScript (TS)* keine Entsprechung hat und daher in dieser Arbeit nicht weiter betrachtet wird.

2.3.3 Entstehungsmechanismen

Bloated Dependencies entstehen durch verschiedene Mechanismen. Bei **direkten** Dependencies tritt häufig evolutionäre Akkumulation auf: Wenn Features umgebaut oder entfernt werden, bleiben die zugehörigen Packages oft im Manifest erhalten [5]. Bei **transitiven** Dependencies ist der Mechanismus fundamentaler: Sie werden automatisch installiert, sobald eine direkte Dependency sie benötigt. Entfernt man später Code, der diese direkte Dependency nutzte, muss die Deklaration manuell aus dem Manifest entfernt werden. Geschieht dies nicht, bleiben alle transitiven Dependencies dieser Dependency installiert, selbst wenn keine davon mehr genutzt wird (vgl. Abschnitt 2.1).

Das nachträgliche Entfernen ist für Entwickler riskant. Statische Analysetools melden zahlreiche False Positives, weil sie dynamische Sprachfeatures wie `eval()`, `require()` mit variablen Parametern oder dynamische Imports nicht vollständig erfassen können. Empirisch zeigt sich, dass Ansätze, die Call-Graph-Analyse mit einer Klassifikation der Dependency-Beziehungen kombinieren, rund ein Drittel dieser False Positives eliminieren können [5]. Eine explorative Studie mit 31 Pull Requests (23 davon beantwortet) deutet an, dass erhebliche Unsicherheiten besonders bei transitiven Dependencies bestehen: Während Entwickler 14 von 15 Vorschlägen zur Entfernung direkter bloated Dependencies akzeptierten, nahmen sie nur 4 von 8 Vorschläge zu transitiven Dependencies an [1].

2.3.4 Empirische Evidenz

Das Problem ist weit verbreitet. Im JavaScript-Ökosystem sind 50,6% aller Dependencies bloated [2]. Im Maven-Ökosystem fällt das Bild noch deutlicher aus: 75,1% aller Dependencies sind bloated, wie eine Analyse von 9.639 Artefakten zeigt [1]. Die Messgrößen unterscheiden sich dabei methodisch: Maven erfasst Dependency-Beziehungen als Kanten im Dependency-Graph (N = 723.444), npm hingegen einzelne Package-Installationen (N = 50.488). Die Prozentwerte sind daher nur bedingt direkt vergleichbar, zeigen aber in beiden Ökosystemen einen hohen Anteil ungenutzter Dependencies. Tabelle 1 und Tabelle 2 zeigen die Verteilung nach Typ.

Typ	**Anteil (N = 723.444)**
Direkt (bloated)	2,7%
Inherited (bloated)	15,4%
Transitiv (bloated)	57,0%
Gesamt bloated	**75,1%**

Tabelle 1: Bloated Dependencies im Maven-Ökosystem [1].

Typ	**Anzahl**	**Anteil der Kategorie**
Direkt (bloated)	120 / 869	13,8%
Indirekt (bloated)	25.446 / 49.619	51,3%
Gesamt bloated	**25.566 / 50.488**	**50,6%**

Tabelle 2: Bloated Dependencies im CommonJS/npm-Ökosystem [2].

Über beide Ökosysteme hinweg zeigt sich ein konsistentes Muster: Transitive Dependencies machen den Großteil des Bloats aus, während direkte Dependencies seltener betroffen sind. Im Maven-Ökosystem existiert zusätzlich die Kategorie geerbter Dependencies aus Parent-POMs (15,4% bloated), die in npm keine Entsprechung hat. Auf Projektebene ist die Verbreitung besonders hoch: 86,2% aller Maven-Artefakte und 98,9% aller untersuchten CommonJS-Packages enthalten mindestens eine bloated-transitive Dependency [1], [2].

Die Hebelwirkung des Entfernens lässt sich am Package `podcast-search` verdeutlichen [2]. Das Projekt hat zwei direkte Dependencies, von denen `npm` zur Laufzeit nie verwendet wird. Alle 679 transitiven Dependencies stammen ausschließlich von dieser einen ungenutzten Dependency. Das Entfernen einer einzigen Zeile in der `package.json` eliminiert so 680 von insgesamt 681 installierten Dependencies, ohne Funktionsverlust.

2.3.5 Konsequenzen

Dependency Bloat hat technische, wirtschaftliche und sicherheitskritische Folgen. Überflüssige Packages erhöhen die Binary-Größe, verlängern Installations- und Build-Zeiten und steigern den Speicherbedarf. In containerisierten Deployments, wo TypeScript-Anwendungen typischerweise betrieben werden, wird dies besonders sichtbar: Analysen offizieller Docker-Images zeigen, dass alle untersuchten Images verwundbare npm-Packages enthalten, im Durchschnitt 16,6 Security-Vulnerabilities pro Container [8].

Dependency-Scanner erkennen diese Vulnerabilities unabhängig von der tatsächlichen Erreichbarkeit des betroffenen Codes. Sicherheitskritisch ist dabei vor allem die vergrößerte Angriffsfläche: Jede zusätzlich installierte Bibliothek kann bekannte Schwachstellen (CVEs) einbringen. SBOM-basierte Scanner erfassen eine Bibliothek als Ganzes, ohne zu berücksichtigen, welche ihrer Funktionen tatsächlich aufgerufen werden. Ponta et al. zeigen, dass erst eine code-zentrische, nutzungsbasierte Analyse feststellen kann, ob vulnerable Funktionen im konkreten Anwendungskontext tatsächlich erreichbar sind [4]. Eine bloated Dependency mit einer bekannten Schwachstelle löst daher eine Warnung aus, obwohl kein Codepfad die Schwachstelle je erreichen könnte. Die Reduktion von Dependency Bloat ist damit nicht nur eine Maintenance-Maßnahme, sondern eine Voraussetzung für präziseres Vulnerability-Management.

2.4 Vulnerability-Datenbanken und Dependency Scanner

Dieses Kapitel beschreibt den Aufbau der Schwachstellendatenbanken, auf denen Dependency-Scanner basieren, und den konkreten Prüfprozess, der zu einem Scanner-Befund führt. Beides bildet die technische Grundlage, um nachzuvollziehen, warum metadatenbasierte Scanner nicht zwischen erreichbaren und nicht erreichbaren Schwachstellen unterscheiden können.

2.4.1 CVE und die National Vulnerability Database

Das *Common Vulnerabilities and Exposures (CVE)*-Programm ist der zentrale Standard zur einheitlichen Benennung öffentlich bekannter Sicherheitslücken [15]. Jede Schwachstelle erhält einen eindeutigen Bezeichner in der Form CVE-JAHR-NUMMER, der eine ökosystemübergreifende Referenzierung ermöglicht. Die *National Vulnerability Database (NVD)* des *National Institute of Standards and Technology (NIST)* baut auf diesem Standard auf und reichert jeden *CVE*-Eintrag mit strukturierten Metadaten

an: einer Schweregradbewertung nach *Common Vulnerability Scoring System (CVSS)*, einer Schwachstellentyp-Klassifikation nach *Common Weakness Enumeration (CWE)* sowie einer Liste betroffener Produkte nach *Common Platform Enumeration (CPE)* [16]. Anwar et al. erfassten in einem Snapshot von Mai 2018 über 107.200 *CVE*-Einträge, die der *NVD* über zwei Jahrzehnte hinzugefügt wurden [16].

CPE ist ein standardisiertes Namensschema zur eindeutigen Identifikation von Software-Produkten und -Versionen. Der *CPE*-Eintrag ist für *NVD*-basierte Dependency-Scanner das entscheidende Matching-Feld: Er codiert betroffene Produkte im Format `cpe:2.3:a:HERSTELLER:PRODUKT:VERSION` und definiert damit, welche Package-Versionen als verwundbar gelten [16]. Allerdings verwenden *CPE*-Bezeichner eine andere Granularität und Konvention als Package-Repository-Koordinaten, was zu ungenauem Package-Mapping führt [9]. Metadatenbasierte Scanner prüfen zudem nicht, ob die vulnerable Funktion einer Bibliothek im konkreten Anwendungskontext tatsächlich aufgerufen wird; das ist der zentrale Unterschied zu code-zentrischen Ansätzen [3].

2.4.2 Von der Datenbank zum Scanner-Befund

Software Composition Analysis (SCA)-Tools wie `npm audit`, das in *Node Package Manager (npm)* integrierte Kommandozeilenwerkzeug zur Sicherheitsprüfung von Abhängigkeiten, oder *Open Worldwide Application Security Project (OWASP)* Dependency-Check, ein verbreitetes Open-Source-Tool der Open Worldwide Application Security Project Foundation, erzeugen automatisch ein Inventar aller eingesetzten Open-Source-Komponenten und gleichen dieses gegen Schwachstellendatenbanken ab [17]. Dabei unterscheiden sich die genutzten Datenquellen je nach Tool: *OWASP* Dependency-Check bezieht Schwachstellendaten unter anderem aus der *NVD* und nutzt *CPE*-Matching zur Identifikation betroffener Packages [17], während `npm audit` seit Oktober 2021 auf die GitHub Advisory Database mit Package-URL (purl) als Identifikator setzt [18]. Trotz unterschiedlicher Datenquellen lässt sich der Ablauf auf drei grundlegende Schritte reduzieren [17]:

1. **Inventarisierung:** Der Scanner liest Dependency-Manifest-Dateien wie `package-lock.json` und erzeugt daraus eine Liste aller Packages mit exakten Versionen [17]. Alternativ kann ein *Software Bill of Materials (SBOM)* (vgl. Abschnitt 2.2) als Eingabequelle dienen, das den in Abschnitt 2.1 beschriebenen Dependency-Tree in maschinenlesbarer Form abbildet.

2. **Matching:** Jedes Package wird gegen die Einträge der jeweiligen Datenbank abgeglichen. Ein Treffer liegt vor, wenn Paketname und Version in den betroffenen Versionsbereich eines Schwachstellen-Eintrags fallen.
3. **Meldung:** Jeder Treffer wird als Befund ausgegeben, inklusive *CVE*-ID, *CVSS*-Score und betroffener Version.

Abbildung 1: Dreistufiger Prüfprozess eines metadatenbasierten Dependency-Scanners (Inventarisierung, Matching, Meldung).

Ausschließlich Paketname und Versionsnummer bestimmen das Ergebnis. Ob die vulnerable Funktion im konkreten Anwendungskontext erreichbar ist, geht in diesen Prozess nicht ein (vgl. die in Abschnitt 2.3 quantifizierten Limitierungen). Genau diese Lücke adressiert der *Proof of Concept (PoC)* dieser Arbeit.

3 Verwandte Arbeiten

3.1 Dynamische Ansätze (Runtime-Tracing, Coverage)

3.2 Statische Ansätze (Call-Graph, Reachability)

3.3 Einordnung der Ansätze nach L0–L2 und Eignung für TypeScript/Node.js

4 Konzept und Designentscheidung

Dieses Kapitel entwickelt das konzeptuelle und methodische Fundament des PoC in drei aufeinander aufbauenden Schritten. Zunächst wird ein gestuftes Evidenzmodell eingeführt, das die in Abschnitt 2.4 und Abschnitt 2.3 beschriebenen Limitierungen metadatenbasierter Scanner formalisiert; es bildet den Bewertungsrahmen für alle nachfolgenden Entscheidungen. Auf dieser Grundlage werden die funktionalen Anforderungen an den PoC abgeleitet und die methodische Entscheidung für eine statische Erreichbarkeitsanalyse (L2a) begründet. Abschließend wird die Gesamtarchitektur der Analysepipeline beschrieben, die diese Entscheidungen in eine konkrete Implementierungsstruktur überführt.

4.1 Usage-Evidence-Schema

Das Schema unterscheidet drei Evidenzstufen: L0 (Präsenz), L1 (Import) und L2 (Erreichbarkeit), die in einer priorisierungsrelevanten Implikationskette vereint sind. Die zentrale These lautet: Metadatenbasierte Scanner operieren auf Präsenzbasis; da sie nicht zwischen Präsenz und Exploitierbarkeit unterscheiden, erzeugen sie strukturell einen überhöhten Befundsatz. Eine stufenweise Evidenzerhebung nach Import- und Erreichbarkeitskriterien erhöht mit jeder Stufe die Wahrscheinlichkeit, dass eine gemeldete Dependency zur Laufzeit nicht ausgeführt wird und ihre Schwachstelle damit strukturell nicht exploitierbar ist. Auf dieser Grundlage können Vulnerability-Befunde risikobasiert priorisiert werden. Diese These verpflichtet den *PoC* zur Implementierung von L1 und L2a (Kapitel 5), die Evaluation zur quantitativen Messung des stufenweisen Gewinns (Kapitel 6) und die Priorisierungslogik zur expliziten Verwendung der Stufenzugehörigkeit als Rankingkriterium.

Die nachfolgende Literaturauswahl erhebt keinen Anspruch auf Vollständigkeit, illustriert jedoch durch unabhängige Forschungsarbeiten konsistent belegte Befunde auf drei verschiedenen Betrachtungsebenen. Ponta et al. formulieren das Grundproblem konzeptionell: Ob eine verwundbare Funktion tatsächlich aufgerufen wird, lässt sich ausschließlich durch eine nutzungsbasierte Analyse feststellen, wobei „nutzungsbasiert„ bedeutet, dass nicht der bloße Abgleich von Paketname und Version, sondern der tatsächliche Aufruf einer Funktion im Anwendungskontext als Maßstab gilt [4]. Dieselbe Forschungsgruppe zeigt, dass 88,8% der von *OWASP* Dependency-Check gemeldeten Befunde *False Positive (FP)*s sind, gemessen an der statischen Aufruf-Erreichbarkeit

der verwundbaren Funktion [3]. Imtiaz et al. bestätigen dies mit einem unabhängigen Werkzeug: In ihrer vergleichenden *SCA*-Tool-Analyse zeigt Steadys statische Analyse, dass bei 84,2% der gemeldeten Befunde das betroffene Package im Anwendungskontext nicht genutzt wird, wobei „genutzt“ bedeutet, dass kein statischer Aufrufpfad zu einer Funktion des Packages existiert [17]. Beide Erhebungen operieren damit auf L2a-Niveau und quantifizieren die Diskrepanz zwischen L0- und L2a-Befundsätzen. Pashchenko et al. messen eine verwandte, aber konzeptuell eigenständige Dimension: den Unterschied zwischen installierten und tatsächlich deployten Dependencies. Sie zeigen, dass diese Unterscheidung in beobachteten Projekten die Anzahl der Warnungen für Development-only-Dependencies gegenüber den Laufzeit-Dependencies um bis zu das Dreifache reduziert [9]. Damit motivieren sie die L0-Grenze selbst: Bereits auf Präsenzebene ist unklar, ob eine installierte Dependency überhaupt im Produktivkontext aktiv ist. Diese drei Studien belegen unabhängig voneinander, dass das Problem der Überschätzung durch Präsenzanalyse auf verschiedenen Abstraktionsebenen auftritt: von der Funktionsebene (Ponta) über die Package-Ebene (Imtiaz) bis zur Deployment-Ebene (Pashchenko). Ein Modell, das zumindest zwischen Präsenz und Erreichbarkeit differenziert, ist damit durch die Forschungslage motiviert; die Stufung als dreistufige Hierarchie ist darüber hinaus durch die unterschiedlichen Analyseebenen der Studien nahegelegt. Mögliche Gegenevidenz, etwa Ökosysteme mit hohem Anteil dynamischer Imports, in denen L1 systematisch an seine Grenzen stößt, wird in der Evaluation (Kapitel 6) als Bedrohung der externen Validität behandelt.

Bevor das Schema formal eingeführt wird, ist eine Abgrenzung zu bestehenden Import-Analyse-Werkzeugen notwendig, um den eigenständigen Beitrag dieser Arbeit zu verorten. Bestehende Werkzeuge wie `depcheck` [19], `knip` [20] und `ts-prune` [21] setzen Import-Analyse als Maintenance-Funktion bereits implizit um. Diese Werkzeuge zeigen, dass Import-Analyse technisch machbar ist; für den Sicherheitskontext dieser Arbeit sind jedoch vier zusätzliche Eigenschaften konstitutiv, die keines dieser Werkzeuge aufweist: erstens eine formale Definition von L1 mit explizitem Geltungsbereich und Ausnahmeklassen; zweitens die Verknüpfung von Import-Evidenz mit Vulnerability-Priorisierung, die diese Werkzeuge nicht leisten, da sie ohne Vulnerability-Bezug operieren [2]; drittens die Differenzierung von L2 in L2a und L2b, die bestehende Forschungsansätze präziser einordnet; und viertens die in der einschlägigen Literatur bisher fehlende[1] Messung des L0→L1-Gaps in einem Sicherheitskontext, die in Kapitel

[1] Die Formulierung bezieht sich auf den Stand der im Rahmen dieser Arbeit konsultierten Literatur; eine vollständige systematische Literaturrecherche wurde nicht durchgeführt.

6 durchgeführt wird. L0 und L2 sind in der Forschungsliteratur verankert [4], [22]; der Beitrag dieser Arbeit liegt damit nicht in der Erfindung der Konzepte, sondern in der formalen Definition von L1 als eigenständiger Evidenzstufe mit expliziten Geltungsbedingungen sowie in ihrer Einbettung in eine priorisierungsrelevante Implikationskette. Damit beantwortet dieser Abschnitt die leitende Frage, unter welchen formal definierten Bedingungen Import-Evidenz als eigenständige Sicherheitsschwelle zwischen Präsenz- und Erreichbarkeitsanalyse operieren kann. Kapitel 3 ordnet bestehende Ansätze anhand dieses Schemas ein.

4.1.1 Terminologie

Um die Implikationskette formal beschreiben zu können, sind vier Begriffe zu klären, die im Schema durchgehend verwendet werden und deren Abgrenzung für die Korrektheit der Kette entscheidend ist. Das Schema ist konzeptuell auf alle Laufzeitumgebungen übertragbar, die den ECMAScript-Modulstandard implementieren [23], darunter Deno, Bun und Browser-Umgebungen. Der *PoC* dieser Arbeit ist jedoch auf TypeScript/ Node.js-Projekte beschränkt, da die verwendeten Analyse-Tools und die verfügbare Forschungslage auf dieses Ökosystem ausgerichtet sind.

Als **Konstrukt** gilt jede referenzierbare Einheit des Laufzeit-relevanten Interfaces einer Dependency, also exportierte Funktionen, Klassen, Objekte und Konstanten. TypeScript-`type`- und `interface`-Deklarationen sind ausgeschlossen, da sie ausschließlich zur Compile-Zeit existieren und kein Laufzeitverhalten erzeugen. Im Kontext dieses Schemas, das auf TypeScript/Node.js-Projekte beschränkt ist, gelten TypeScript-Dekoratoren als Konstrukte im obigen Sinne. Grund ist ihre Laufzeitsemantik: Sie werden über den Reflection-Mechanismus ausgeführt und können dabei potenziell verwundbare Codepfade aktivieren.

Als **Anwendungscode** gilt der Teil des Projekts außerhalb des `node_modules`-Verzeichnisses[2].

Als **Side-Effect-Import** gilt ein Import der Form `import 'modulname'` ohne Named- oder Default-Import. Ein typisches Beispiel ist `import 'reflect-metadata'`, das in NestJS-Projekten verwendet wird, um den Reflection-Mechanismus für Dekoratoren zu aktivieren: Das Modul wird beim Laden sofort ausgeführt und registriert globale

[2]Der Begriff `node_modules` bezeichnet hier stellvertretend das jeweilige Paketverzeichnis der Laufzeitumgebung. In Deno etwa entspricht dies dem globalen Cache-Verzeichnis, in Bun dem `.bun` -Verzeichnis. Der Geltungsbereich des Schemas umfasst konzeptuell alle ECMAScript-konformen Laufzeitumgebungen [23]; die Benennung folgt der im *PoC* verwendeten Node.js-Konvention.

Metadaten, ohne dass ein Konstrukt im obigen Sinne exportiert wird. Side-Effect-Imports sind L1-positiv, können aber nicht auf L2-Niveau eingeordnet werden, da kein adressierbares Konstrukt für die Aufrufstrukturanalyse identifizierbar ist. Das Schema kann damit nicht differenzieren, ob der Top-Level-Code des Moduls verwundbare Pfade aktiviert oder nicht. Da im Sicherheitskontext ein nicht erkannter verwundbarer Pfad (*False Negative (FN)*) schwerwiegender ist als ein fälschlich gemeldeter Befund (*FP*), entsprechend dem Principle of Fail-Safe Defaults, das Saltzer und Schroeder als eines von acht grundlegenden Sicherheitsprinzipien für Systemdesign formulieren [24], werden Side-Effect-Imports konservativ als hoch priorisiert. Die Behandlung in der Priorisierungslogik wird in Kapitel 5 erläutert.

Als ***Dependency Injection (DI)*-Mechanismus** gilt jedes Verfahren, bei dem Abhängigkeiten nicht über eine explizite `import`- oder `require()`-Anweisung referenziert werden, sondern zur Laufzeit automatisch durch ein Framework aufgelöst werden. In NestJS etwa wird eine `DatabaseService`-Klasse als Konstruktor-Parameter deklariert und vom *Inversion of Control (IoC)*-Container automatisch instanziiert, ohne dass ein expliziter `require()`-Aufruf im Quellcode des konsumierenden Moduls erscheint [25]. Da der Modulpfad dabei im Quellcode nicht sichtbar ist, entziehen sich *DI*-geladene Dependencies der L1- und L2a-Analyse.

4.1.2 Formaler Rahmen und Geltungsbereich

Die beschriebenen Teilmengenrelationen lassen sich als Trichtermodell visualisieren, das die echte Inklusion der Evidenzstufen verdeutlicht (Abbildung 2): Die Menge der importierten Dependencies ist eine echte Teilmenge der installierten, und die Menge der erreichbaren Konstrukte ist eine echte Teilmenge der importierten.

Abbildung 2: Trichtermodell: Jede Evidenzstufe beschreibt eine echte Teilmenge der vorherigen.

Das Trichtermodell gilt unter einer zentralen Annahme, die den operativen Geltungsbereich des Schemas begrenzt: der Closed-World-Annahme A. Das Schema erhebt keinen Anspruch auf logische Vollständigkeit unter beliebigen Laufzeitbedingungen, sondern formuliert eine operational definierte heuristische Implikationskette L2 → L1 → L0, deren Anwendungsbereich durch A explizit begrenzt ist. A besagt: Alle relevanten Imports sind im statisch analysierbaren Quellcode sichtbar, und kein Modul wird über nicht-sichtbare Mechanismen geladen. Unter A folgt aus L2a-Evidenz L1-Evidenz, und aus L1-Evidenz L0-Evidenz. Vier Ausnahmeklassen können A verletzen: dynamische Imports wie `require(someVariable)`, bei denen der Modulpfad erst zur Laufzeit bekannt ist; Conditional-Requires wie `if (process.env.DEBUG) require('debug-lib')`, die nur unter bestimmten Laufzeitbedingungen ausgeführt werden; *DI*-Mechanismen, die Abhängigkeiten ohne explizite Import-Anweisung zur Laufzeit auflösen; und `peerDependencies`, bei denen ein Package eine Laufzeit-Abhängigkeit deklariert, die vom Host-Projekt bereitgestellt wird, ohne einen eigenen Import-Statement zu erzeugen. Ein Beispiel ist ein ESLint-Plugin, das `eslint` als `peerDependency` deklariert und zur Laufzeit nutzt, ohne `require('eslint')` im eigenen Quellcode zu schreiben [26], [27]. Die Gültigkeit der Implikationskette setzt Annahme A voraus; ihre Grenzen werden in Kapitel 6 als Bedrohungen der externen Validität operationalisiert. Für den in dieser Arbeit verwendeten Testkorpus, der keine *DI*-lastigen Projekte oder Plugin-Architekturen enthält, ist die praktische Relevanz dieser Ausnahmeklassen begrenzt. Sie werden dennoch vollständig in Kapitel 6 dokumentiert.

`writex` ist ein *npm*-Tool zum Konvertieren von Markdown-Dateien in LaTeX, das Nielsen et al. als Motivationsbeispiel für die Unzulänglichkeit von *npm* audit verwenden [22]. Es wird hier als durchgehendes Beispiel übernommen, um den Erkenntnisgewinn jeder Evidenzstufe direkt vergleichbar zu machen. Die drei Evidenzstufen werden nun anhand dieses Beispiels sequenziell durchlaufen.

L0 (Präsenz) bildet die Baseline des Schemas. Da Scanner ausschließlich auf Paketname und Version prüfen, werden Dependencies als verwundbar gemeldet, unabhängig davon, ob ihr Code je ausgeführt wird. Für `writex` meldet *npm* audit 10 Vulnerabilities aus 5 Advisories, darunter eine Prototype-Pollution-Schwachstelle in `lodash`, die gleich zweimal gemeldet wird, weil `lodash` über zwei verschiedene Dependency-Chains installiert ist: `writex → lodash-template-stream → lodash` und `writex → gaze → globule → lodash` [22]. Insgesamt sind 53 Packages installiert, einschließlich aller transitiven Dependencies, die durch den in Abschnitt 2.1 beschriebenen

Kaskadeneffekt installiert wurden. L0 meldet alle davon als potenziell betroffen, ohne zu unterscheiden, welche tatsächlich geladen werden. Auf dieser Stufe lässt sich am wenigsten eingrenzen, ob eine gemeldete Dependency zur Laufzeit tatsächlich ausgeführt wird.

L1 (Import) verfeinert L0 durch eine zentrale Sicherheitsbeobachtung: Jede ECMAScript-konforme Laufzeitumgebung initialisiert ein Modul genau dann, wenn es über einen expliziten `import`- oder `require()`-Aufruf geladen wird [23], [28]. Eine Dependency, die nie importiert wird, wird nie initialisiert; auch ihre potenziell verwundbaren Funktionen können daher strukturell nicht ausgeführt werden. Damit markiert L1 im Schema die erste Evidenzstufe mit einer sicherheitsrelevanten Implikation: Nicht-Laden schließt einen Ausführungspfad strukturell aus. Fehlt der Import-Nachweis, steigt die Wahrscheinlichkeit, dass die Dependency zur Laufzeit nicht ausgeführt wird und eine gemeldete Schwachstelle strukturell nicht exploitierbar ist. Unter Annahme A bildet die Menge der L1-positiven Dependencies eine obere Schranke der zur Laufzeit geladenen Dependencies: Jede Dependency, die nicht importiert wird, kann nicht ausgeführt werden. Für `writex` bedeutet das: Von den 53 installierten Packages umfassen deren JavaScript-Quelldateien insgesamt 187 Module; L1 filtert alle Packages heraus, die in keiner dieser Moduldateien per Import-Statement referenziert werden. Diese Garantie gilt für den Geltungsbereich dieser Evaluation; auf beliebige TypeScript/Node.js-Projekte ist sie nicht übertragbar, da dynamische Imports, `peerDependencies` oder *DI*-Mechanismen Annahme A verletzen können (vgl. Ausnahmeklassen in Abschnitt 4.1.2).

Die wesentliche Einschränkung von L1 gegenüber L2a besteht darin, dass L1 nicht unterscheiden kann, ob eine importierende Datei selbst vom Einstiegspunkt der Anwendung erreichbar ist. Eine Datei `legacy-convert.ts`, die `lodash` importiert, aber vom Einstiegspunkt von `writex` nie aufgerufen wird, klassifiziert `lodash` als L1-positiv, obwohl zur Laufzeit kein Ausführungspfad von diesem Import ausgeht. L1 garantiert, dass nicht-importierte Dependencies keine Laufzeit-Relevanz besitzen. Gleichwohl überschätzt L1 die tatsächliche Nutzung systematisch: Es kann nicht ausgeschlossen werden, dass eine importierende Datei selbst vom Einstiegspunkt der Anwendung unerreichbar ist. Genau diese Lücke schließt L2a durch eine Erreichbarkeitsanalyse vom Einstiegspunkt.

Als formale Sicherheitsevidenzstufe ist L1 in der Forschungsliteratur nicht beschrieben, wie im vorigen Absatz dargelegt. Werkzeuge wie `depcheck`, `knip` und `ts-prune` analysieren Import-Statements als Maintenance-Werkzeuge ohne Vulnerability-Bezug und ohne definierten Geltungsbereich [2]; Nielsen et al. springen direkt von L0 zu L2a,

ohne eine Import-Stufe zu formalisieren [22]. Der isolierte Sicherheitsmehrwert von L1 gegenüber L0 wird in der Evaluation (Kapitel 6) erstmals gemessen.

L2 (Erreichbarkeit) schließt die verbleibende Lücke von L1. **L2a** bezeichnet den durch Aufrufstrukturanalyse nachweisbaren Pfad vom Einstiegspunkt zu einem Konstrukt. Fehlt dieser Pfad, erhöht sich die Wahrscheinlichkeit weiter, dass die Dependency zur Laufzeit nicht ausgeführt wird: Eine Dependency, die zwar importiert, aber vom Einstiegspunkt aus nie aufgerufen wird, besitzt keinen nachweisbaren Ausführungspfad über Anwendungslogik. Für `writex` zeigt die Aufrufstrukturanalyse, dass von den 187 Modulen nur 90 tatsächlich vom Einstiegspunkt aus erreichbar sind, was 42 von 53 Packages entspricht [22]. Von den 10 ursprünglichen *npm* audit-Befunden verbleibt nach L2a-Analyse genau einer als True Positive: eine *Regular Expression Denial of Service (ReDoS)*-Schwachstelle in der Funktion `minimatch(path, pattern)` des `minimatch`-Packages, die über die Kette `writex → globule → minimatch` erreichbar ist. Die anderen 4 Advisories (9 Befunde) sind *FP*s, weil die verwundbaren Funktionen, darunter die `lodash`-Prototype-Pollution, vom Anwendungscode nie aufgerufen werden [22]. Nielsen et al. berichten für L2a eine *FP*-Reduktion von ca. 81% gegenüber L0 (N=12; ohne Konfidenzintervall), die hier als Orientierungsgröße dient. Dieser Einschränkung begegnet die größere Studie von Zhou et al.: Sie messen für *SBOM*-basierte Scanner über N=2.414 Repositories eine *FP*-Rate von 97,5% (entsprechend einer Präzision von ca. 2,5%) sowie eine *FP*-Reduktion von 63,3% durch Call-Graph-Reachability [29], was zeigt, dass das Problem nicht auf den Stichprobenumfang von Nielsen et al. (N=12) zurückzuführen ist.

L2b (dynamisch/konzeptuell) bezeichnet durch Laufzeitbeobachtung bestätigte Ausführung: Selbst wenn ein Aufrufpfad zu `minimatch` existiert, bleibt bei L2a offen, ob dieser Pfad in einer konkreten Ausführung tatsächlich durchlaufen wird. L2b würde dies durch Instrumentierung zur Laufzeit klären. Im *PoC* dieser Arbeit ist L2b nicht implementiert, da die erforderliche Laufzeitbeobachtung eine kontrollierte Deployment-Umgebung voraussetzt, die den Rahmen dieser Arbeit übersteigt. Die methodischen Ansätze beider Unterebenen werden in den Verwandten Arbeiten (Kapitel 3) eingeordnet.

Tabelle 3 fasst alle vier Stufen als Überblick zusammen. Die Spalte ***FN*-Klasse** beschreibt, welche Arten von Dependencies eine Stufe strukturell nicht erfassen kann. Die Spalte **Empirischer Mehrwert** gibt an, welchen messbaren Vorteil eine Stufe gegenüber L0 liefert. Bei L1 fehlt dieser Wert in der Literatur; die Messung in Kapitel

6 schließt diese Lücke. L2b ist als konzeptueller Ausblick aufgenommen; die Implementierung ist aus den oben genannten Gründen nicht Teil des *PoC* und wird in Kapitel 7 als zukünftige Arbeit aufgegriffen.

Stufe	**Frage**	**Methode**	***FN*-Klasse**	**Empirischer Mehrwert**
L0 (Präsenz)	Ist die Dependency installiert?	*SBOM*, Dependency-Tree (vgl. Abbildung 1)	Keine *FN*s (alle installierten Packages erfasst)	Baseline; *SBOM*-Scanner: 2,5% Präzision (N=2.414) [29]
L1 (Import)	Wird die Dependency geladen?	Statische Quellcodeanalyse der Imports	Dynamische Imports, peerDependencies, *DI*[3]	Bedingte obere Schranke; erstmals in Kap. 6 erhoben
L2a (statisch)	Ist ein Konstrukt erreichbar?	Aufrufstrukturanalyse (vgl. Kapitel 3)	Reflection, *DI*, globale Augment.[4]	81% / 63,3% *FP*-Red. ggü. L0 (vgl. L2a-Beschr.)
L2b (dynamisch) **(konzeptuell)**	Wird ein Konstrukt ausgeführt?	Laufzeitbeobachtung (vgl. Kapitel 3)	Unvollständige Testabdeckung	Präziseste Evidenz; nicht impl. (vgl. Kapitel 7)

Tabelle 3: Evidenzstufen des Usage-Evidence-Schemas als Überblick. L1 wird in dieser Arbeit erstmals als eigenständige Sicherheitsstufe formalisiert. Die zugehörige Messung erfolgt in Kapitel 6. L2b erfordert eine kontrollierte Laufzeitumgebung; seine Umsetzung wird in Kapitel 7 als Gegenstand künftiger Forschung diskutiert.

Die technische Umsetzung der risikobasierten Priorisierungslogik wird in Kapitel 5 beschrieben; ihre Validierung erfolgt in Kapitel 6.

[3] Zu den Ausnahmeklassen, die Annahme A verletzen, vgl. Abschnitt 4.1.2.

[4] Augmentierungen bezeichnen das nachträgliche Erweitern globaler Objekte zur Laufzeit, z.B. `Object.prototype.sanitize = ...`.

4.2 Anforderungen an den PoC

4.3 Bewertung und Methodenauswahl

4.4 Architektur der Gesamtpipeline

5 Implementierung

5.1 Tooling-Stack und Projektstruktur

5.2 L0: Präsenzermittlung via Dependency-Tree/SBOM

5.3 L1: Loaded-Heuristiken

5.4 L2: Reachability-Analyse (Call-Graph, Entry Points, Mapping)

5.5 Ergebnisformat und Risk-Scoring

5.6 Beispiel-Durchlauf

6 Evaluation

6.1 Setup und Testkorpus

6.2 Metriken und Vorgehen

6.3 Ergebnisse und Vergleich mit npm audit

6.4 Limitierungen (JS-Dynamik, Reflection, Bundling)

7 Fazit und Ausblick

7.1 Fazit

7.2 Ausblick

Literaturverzeichnis

[1] C. Soto-Valero, M. Monperrus, und B. Baudry, „A comprehensive study of bloated dependencies in the Maven ecosystem“, *Empirical Software Engineering*, Bd. 26, Nr. 4, S. 1–36, 2021, doi: 10.1007/s10664-020-09914-8.

[2] Y. Liu, D. Tiwari, C. Bogdan, und B. Baudry, „Detecting and removing bloated dependencies in CommonJS packages“, *Journal of Systems and Software*, Bd. 230, S. 112509, Dez. 2025, doi: 10.1016/j.jss.2025.112509.

[3] S. E. Ponta, H. Plate, und A. Sabetta, „Detection, assessment and mitigation of vulnerabilities in open source dependencies“, *Empirical Software Engineering*, Bd. 25, Nr. 5, S. 3175–3215, 2020, doi: 10.1007/s10664-020-09830-x.

[4] S. E. Ponta, H. Plate, und A. Sabetta, „Beyond Metadata: Code-Centric and Usage-Based Analysis of Known Vulnerabilities in Open-Source Software“, in *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2018, S. 449–460. doi: 10.1109/ICSME.2018.00054.

[5] C.-C. Chuang, L. Cruz, R. van Dalen, V. Mikovski, und A. van Deursen, „ Removing dependencies from large software projects: are you really sure? “, in *2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM)*, Los Alamitos, CA, USA: IEEE Computer Society, Okt. 2022, S. 105–115. doi: 10.1109/SCAM55253.2022.00017.

[6] C. Soto-Valero und others, „A comprehensive study of bloated dependencies in the Maven ecosystem“, *Empirical Software Engineering*, 2021, doi: 10.1007/s10664-020-09914-8.

[7] C. Soto-Valero, T. Durieux, N. Harrand, und B. Baudry, „Coverage-based debloating for java bytecode“, *ACM Transactions on Software Engineering and Methodology*, Bd. 32, Nr. 2, S. 1–34, 2023.

[8] A. Zerouali, V. Cosentino, T. Mens, G. Robles, und J. M. Gonzalez-Barahona, „On the Impact of Outdated and Vulnerable Javascript Packages in Docker Images“, in *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2019, S. 619–623. doi: 10.1109/SANER.2019.8667984.

[9] I. Pashchenko, H. Plate, S. E. Ponta, A. Sabetta, und F. Massacci, „Vuln4Real: A Methodology for Counting Actually Vulnerable Dependencies“, *IEEE Transac-*

tions on Software Engineering, Bd. 48, Nr. 5, S. 1592–1609, 2022, doi: 10.1109/TSE.2020.3025443.

[10] A. Decan, T. Mens, und M. Claes, „An empirical comparison of dependency network evolution in seven software packaging ecosystems“, *Empirical Software Engineering*, Bd. 24, Nr. 1, S. 381–416, 2019, doi: 10.1007/s10664-017-9589-y.

[11] R. Kikas, R. Söre, M. Dumas, und D. Pfahl, „Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem“, in *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, ACM, 2022, S. 220–231. doi: 10.1145/3510003.3510142.

[12] National Telecommunications and Information Administration, „Software Bill of Materials (SBOM)“. 2021.

[13] National Telecommunications and Information Administration, „The Minimum Elements for a Software Bill of Materials (SBOM)“. [Online]. Verfügbar unter: https://www.ntia.gov/files/ntia/publications/sbom_minimum_elements_report.pdf

[14] International Organization for Standardization und International Electrotechnical Commission, „Information technology — SPDX®“. [Online]. Verfügbar unter: https://www.iso.org/standard/81870.html

[15] CVE Program, „Common Vulnerabilities and Exposures (CVE)“. 2024.

[16] A. Anwar, A. Abusnaina, S. Chen, F. Li, und D. Mohaisen, „Cleaning the NVD: Comprehensive Quality Assessment, Improvements, and Analyses“, *IEEE Transactions on Dependable and Secure Computing*, Bd. 19, Nr. 5, S. 3368–3382, 2022, doi: 10.1109/TDSC.2021.3125270.

[17] N. Imtiaz, S. Thorn, und L. Williams, „A Comparative Study of Vulnerability Reporting by Software Composition Analysis Tools“, in *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, in ESEM '21. New York, NY, USA: Association for Computing Machinery, 2021, S. 1–11. doi: 10.1145/3475716.3475769.

[18] GitHub, „GitHub Advisory Database Now Powers npm audit“. [Online]. Verfügbar unter: https://github.blog/security/supply-chain-security/github-advisory-database-now-powers-npm-audit/

[19] depcheck contributors, „depcheck: Check your npm module for unused dependencies“. [Online]. Verfügbar unter: https://github.com/depcheck/depcheck

[20] A. Boshuizen, „knip: Find unused files, exports and dependencies in your project“. [Online]. Verfügbar unter: https://knip.dev/

[21] T. Spencer, „ts-prune: Find potentially unused exports in your TypeScript project“. [Online]. Verfügbar unter: https://github.com/nadeesha/ts-prune

[22] B. B. Nielsen, M. T. Torp, und A. Møller, „Modular call graph construction for security scanning of Node.js applications“, in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, in ISSTA 2021. Virtual, Denmark: Association for Computing Machinery, 2021, S. 29–41. doi: 10.1145/3460319.3464836.

[23] Ecma International, „ECMAScript 2024 Language Specification (ECMA-262)“. [Online]. Verfügbar unter: https://tc39.es/ecma262/

[24] J. H. Saltzer und M. D. Schroeder, „The Protection of Information in Computer Systems“, *Proceedings of the IEEE*, Bd. 63, Nr. 9, S. 1278–1308, 1975.

[25] NestJS Contributors, „NestJS Documentation: Dependency Injection“. [Online]. Verfügbar unter: https://docs.nestjs.com/fundamentals/custom-providers

[26] T. Zimmermann, T. F. Bissyandé, B. Danglot, N. Harrand, M. Monperrus, und F. Trautsch, „Not All Dependencies Are Equal: An Empirical Study on Production Dependencies in NPM“, in *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, ACM, 2022, S. 316–327. doi: 10.1145/3524842.3528454.

[27] npm Contributors, „package.json: peerDependencies“. [Online]. Verfügbar unter: https://docs.npmjs.com/cli/v10/configuring-npm/package-json#peerdependencies

[28] Node.js Contributors, „Modules: CommonJS modules“. [Online]. Verfügbar unter: https://nodejs.org/api/modules.html

[29] Y. Zhou, X. Hu, Z. Chen, T. Xu, und X. Yang, „A Reality Check on SBOM-based Vulnerability Management: An Empirical Study and A Path Forward“, *arXiv preprint arXiv:2511.20313*, 2025, doi: 10.48550/arXiv.2511.20313.